PROJECT REPORT · WORKING END-TO-END

NL2SQL Server

A natural-language query layer that turns a plain-English question into SQL, runs it against the right organizational database, and hands back real rows — without the asker ever needing to know the schema.

Status — confirmed working, real questions in / real rows out

Stack — Node.js · TypeScript · Express · Postgres · Claude or a local LLM **Report date** — July 3, 2026

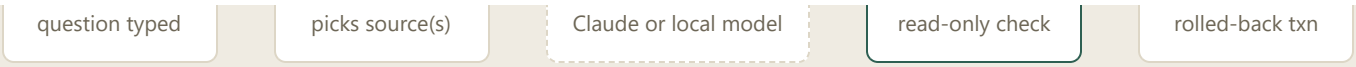
2	173	15 / 15	$0
LLM providers wired in	Tables introspected	Tests passing	Spent to run it end-to-end

What it does

Organizations keep data on multiple servers with different schemas — often different engines and query languages entirely. Answering a question normally requires knowing which server holds the answer and how to write SQL against it. This project removes that requirement: a user types a question in English, and the server figures out where to look, writes the query, checks that it's safe, runs it, and returns the results.

The design target is **federation** — the ability to add a second, third, or Nth server later without rewriting the parts that already work. Today, one data source is registered (a Postgres HR system); the abstraction that lets more join in is already in place and exercised in code, just not yet proven against a second server.

How a question becomes an answer



The `SQL Guard` step is the real trust boundary: the model only ever *proposes* SQL — anything that isn't a single read-only `SELECT`/`WITH` statement is rejected outright, and every query that does run executes inside a transaction that's rolled back afterward, so nothing can be written even if a check were somehow bypassed. If the generated SQL still fails against the real database (e.g. an invented table name), the actual database error is fed back to the model for one self-correction retry before giving up.

The LLM is swappable behind one setting

The dashed `LLM` node is a single function, `generateSql()`, that dispatches to either Claude or a locally-hosted Ollama model based on one environment variable — nothing else in the router, guard, or API layer changes:

```
LLM_PROVIDER=anthropic   // default -- calls the Claude API
LLM_PROVIDER=ollama       // calls a model running on localhost, no API key needed
```

The extensibility point

Every server the org holds data in implements one interface. A future second data source is a new class implementing this same shape — the router, the prompt logic, and the API layer don't change:

```
interface DataSource {
  id: string;
  description: string;   // routing context for the LLM
  dialect: string;        // "postgres", "mysql", ... shapes the SQL prompt
  listTables(): Promise<TableInfo[]>;
  executeReadOnlyQuery(sql: string): Promise<QueryResult>;
}
```

Proof it works

Two real questions, asked against the live restored HR database, answered entirely by the local model — no Claude API call involved, no cost:

Q **List the 5 most recently added employees**

```
SELECT name FROM pers.employee_master
ORDER BY date_of_joining_org DESC LIMIT 5;
```

Result:

`Vijay Nair, Hari Menon, Sunita Rao, Anil Kapoor, Meera Iyer`

Q How many employees are there in total?

```
SELECT COUNT(*) AS total_employees
FROM pers.employee_master WHERE is_active = true;
```

Result:

`965`

Workflow we followed

This wasn't a straight line — the path from "server exists" to "answers real questions offline" involved hitting a real blocker and routing around it. In order:

1. **Started from an existing prototype**

   The Postgres connector, the SQL safety guard, and the Claude wiring already existed. Verified each piece independently before changing anything: schema introspection correctly read all 173 tables, and a direct query against `pers.employee_master` returned real rows.

2. **Tried a live `/ask` call**

   The request reached Claude correctly but came back `400 credit balance too low` — a well-formed error, which itself confirmed the request shape, auth header, and model name were all correct. This was a billing problem, not a code problem.

3. **Investigated funding, found a mismatch**

   A $20 subscription had been paid the day before — but it was **Claude Pro** (claude.ai, the chat website). API access is a separate product billed through **console.anthropic.com** with its own prepaid credits; a Pro subscription doesn't fund it. Decided to skip funding the API entirely and go fully local instead.

4. **Installed a local model runtime**

   Installed **Ollama** and pulled `qwen2.5-coder:7b` (~4.7GB, one-time download) — a free, open-weight model that runs entirely offline on this machine, no account or API key required.

5. **Added a second LLM provider behind one setting**

`generateSql()` now checks `LLM_PROVIDER` and dispatches to either Claude or the local model. The router, the safety guard, and the API layer needed zero changes.

6 **First real run hallucinated a table**

The model invented `utility.employee`, which doesn't exist. Root cause: Ollama silently caps its context window at 2048 tokens by default, truncating the 173-table / ~2,200-column schema before the model ever saw the right table.

7 **Fixed it without more RAM**

This machine has ~8GB RAM with little free, so a bigger context window wasn't really available. Instead, added a lightweight relevance filter that scores tables by keyword overlap with the question and sends only the ~20 most relevant — a much smaller, cheaper prompt.

8 **Added a self-correction retry**

If the generated SQL still fails to execute, the real Postgres error is fed back to the model for one more attempt before giving up — this applies to both the Claude and local paths.

9 **Verified it end-to-end, twice**

Two different real questions, run against the live restored database, both produced correct SQL and correct answers — entirely offline, at no cost. See **Proof it works** above.

Current status

PROVEN

- Schema introspection reads all 173 tables correctly across the `pers`, `digitalsig`, `request`, `training`, and `utility` schemas.
- A complete `/ask` round trip — English question in, generated SQL, real rows out — confirmed live against the local model, twice (see **Proof it works**).
- The self-correction retry works as designed: an early run had the model invent a nonexistent table; the real database error was fed back and the next attempt used the correct one.
- The safety guard rejects a `DELETE` before it ever reaches the database.
- The Express server boots; `/health` responds; the web form at `/` renders and submits correctly.

- The Claude code path was also verified up to the point of calling the Anthropic API — the request reaches Claude and gets a well-formed response, confirming the wiring is correct.

BLOCKED, NOT BROKEN (CLAUDE PATH ONLY)

- The configured Anthropic key has a **$0 credit balance**, so the Claude path itself hasn't produced a real answer yet — every generation request returns `400 credit balance too low`. This doesn't block the project: the local-model path is architecturally identical and fully verified.
- Turned out to be a product mix-up, not a missing payment — the $20 already spent went to a Claude Pro chat subscription, a different product from API credits.
- Funding the API key would make the Claude path work immediately with no code changes — same guard, same connector, same JSON contract.

ASPIRATIONAL

- Multi-server federation: the interface and multi-source routing logic exist and run, but only one source is registered. Proving it means wiring up a second, differently-shaped server.
- Production hardening: no auth on `/ask`, no per-role table allow-list, and the database role has more than read privilege at the database level. A deliberate scope decision for a prototype, not an oversight.

Verification

LAYER	HOW IT WAS CHECKED	RESULT
SQL safety guard	Unit tests — multi-statement SQL, DML/DDL keywords, SELECT/WITH acceptance, LIMIT injection	15 tests, all passing
Postgres connector	Integration tests against the real restored database — list tables, run a read query, reject a write	Passing against live data
Web server	Manual request to `/health` and the static form	Responding correctly
End-to-end `/ask` (local model)	Two real prompts, full round trip to real rows	Both correct, ~40s each

End-to-end `/ask` (Claude)	Manual request with a real prompt	Reaches Claude; blocked on billing only

Running it

```
cd nl2sql-server
npm install
cp .env.example .env   # PG_* connection info; LLM_PROVIDER=ollama by default
npm run dev             # http://localhost:3000

npm test                  # 15/15 passing
```

By default `.env` is set to `LLM_PROVIDER=ollama`, so it needs no Anthropic credit — only Ollama installed with `qwen2.5-coder:7b` pulled and `ollama serve` running. Switch to `LLM_PROVIDER=anthropic` once the API key is funded. The web form at `http://localhost:3000/` is the easiest way to try it — one question box, one results table (expect ~40s per answer on modest CPU-only hardware). The API itself is a single endpoint:

```
curl -X POST http://localhost:3000/ask \
  -H "Content-Type: application/json" \
  -d '{"prompt": "List the 10 most recently added employees"}'
```

Full technical writeup in `PROJECT.md` · source at `nl2sql-server/`